

## Software Requirements Specification (SRS)

System: "The Internet" – UI Component and Web Behavior Demonstration

Site Base URL: [the-internet.herokuapp.com](https://the-internet.herokuapp.com)

### 1. Introduction

#### 1.1 Purpose

The purpose of this SRS is to define the observable functional and non-functional requirements of the public web application hosted at [the-internet.herokuapp.com](https://the-internet.herokuapp.com). The site provides a collection of independent examples demonstrating common web UI components, behaviors, and edge cases, primarily for use in web automation, QA training, and exploratory testing scenarios. The document is based exclusively on behaviors, content, and interactions observable from the deployed site and does not rely on any unverified implementation assumptions.

#### 1.2 Intended Users

The primary intended users inferred from the site content are:

- QA engineers and test automation engineers exploring UI automation scenarios (e.g., dynamic content, frames, JavaScript alerts, broken images, etc.)
- Developers and SDETs (Software Development Engineers in Test) experimenting with locators and asynchronous behaviors
- Trainers, educators, and learners using the site as a demo system for practicing and demonstrating web testing techniques

End-users are assumed to be technically proficient and familiar with basic web navigation.

#### 1.3 Scope

The system consists of:

- A home page listing multiple independent "example" pages and links to them
- A set of self-contained example pages, each demonstrating a specific UI pattern or behavior such as:
  - Static and dynamic content
  - Forms and authentication
  - Alerts and modals
  - Tables and data structures
  - Layout constructs (frames, shadow DOM, large DOM, shifting content)
  - Browser-related behaviors (status codes, multiple windows, context menus, JavaScript errors, geolocation)

Each example page is functionally independent and can be accessed directly via URL or through the homepage navigation.

The scope of this SRS includes:

- All pages and features listed on the home page's "Available Examples" section
- Core observable behaviors of each feature page, including UI elements, user triggered actions, and resulting feedback
- Cross-cutting behaviors such as navigation and layout

The scope excludes any back-end systems not visible from the browser and any undocumented behaviors not observable from the UI.

## 2. System Overview

### 2.1 High-Level Description

"The Internet" application is a static, example-driven site composed of multiple independent demonstration modules. Each module:

- Is accessible via a dedicated URL path, typically `/feature_name` (e.g., `/login`, `/checkboxes`, `/dynamic_controls`)
- Presents descriptive text explaining the example's purpose, often with explicit statements about how content changes across page loads or interactions (e.g., dynamic content, typos, shifting content)
- Provides minimalistic UI controls (links, buttons, inputs, sliders, tables) with intentionally simple layout and styling

The site does not expose any global login, profile, or shared session features; the "secure area" is restricted to the context of the login example only.

### 2.2 Navigation Structure

#### 2.2.1 Home Page

The root URL (`/`) displays:

- A main heading "Welcome to the-internet"
- A section "Available Examples" listing all example titles as individual links
- Each list item's text corresponds to the feature name (e.g., "Checkboxes", "Form Authentication", "Broken Images", "Dynamic Content", "Sortable Data Tables", etc.) and links to the respective example page

#### 2.2.2 Example Pages

Each example is hosted under a unique path, for example:

| Path                     | Description                      |
|--------------------------|----------------------------------|
| <code>/checkboxes</code> | Checkboxes example               |
| <code>/login</code>      | Form authentication / login page |
| <code>/dropdown</code>   | Dropdown selection example       |

| Path                           | Description                        |
|--------------------------------|------------------------------------|
| /dynamic_controls              | Asynchronous element changes       |
| /dynamic_loading               | Dynamically loaded elements        |
| /upload                        | File uploader                      |
| /javascript_alerts             | JS alert/confirm/prompt examples   |
| /drag_and_drop                 | Drag-and-drop example              |
| /tables                        | Sortable data tables               |
| /notification_message_rendered | Random notification messages       |
| /entry_ad                      | Entry advertisement modal          |
| /typos                         | Random typo example                |
| /add_remove_elements/          | Add/remove elements example        |
| /disappearing_elements         | Disappearing menu elements         |
| /hovers                        | Hover behavior with user profiles  |
| /abtest                        | A/B test concept description       |
| /dynamic_content               | Dynamic text and images            |
| /status_codes                  | HTTP status codes information      |
| /inputs                        | Numeric input field                |
| /horizontal_slider             | Slider and displayed value         |
| /context_menu                  | Right-click context menu alert     |
| /challenging_dom               | Complex DOM locators and table     |
| /exit_intent                   | Exit-intent modal                  |
| /jqueryui/menu                 | jQuery UI menu example             |
| /javascript_error              | Page with JavaScript error on load |
| /windows                       | Opening a new window               |
| /large                         | Large and deep DOM example         |
| /download                      | File downloader                    |
| /forgot_password               | Email-only password retrieval form |

| Path             | Description                                                      |
|------------------|------------------------------------------------------------------|
| /geolocation     | "Where am I?" location button                                    |
| /floating_menu   | Floating menu with long scrollable content                       |
| /infinite_scroll | Infinite scroll example                                          |
| /frames          | Entry page to frames examples; links to nested frames and iframe |
| /shadowdom       | Shadow DOM example                                               |

Other linked examples listed on the home page but not fully inspected follow similar patterns and are assumed to be independent demonstration modules, without additional cross-module dependencies.

Each example usually includes a "Powered by Elemental Selenium" footer line.

## 2.3 Assumptions and Constraints

### 2.3.1 Assumptions

- Users access the site via a modern desktop browser capable of executing JavaScript, handling alerts, and supporting HTML5 features (e.g., file uploads, geolocation APIs)
- Users may directly navigate to any example page without visiting the home page first
- Content and behaviors described on each page (e.g., "typo being introduced randomly" or "content loads new text and images on each refresh") reflect the intended run-time behavior

### 2.3.2 Constraints

- The site is read-only with respect to persistent data. No persistent user data management (registration, database-backed profiles) is exposed
- Authentication behavior is limited to the "Form Authentication" example context (login page and secure area) and does not extend globally
- Some behaviors are deliberately non-deterministic or randomized (e.g., dynamic content, random typos, notification message variation), constraining deterministic expectations for exact text on every page load
- Some features depend on browser/OS behavior (e.g., context menu, geolocation, multiple windows, alert dialogs); exact system dialogs and browser chrome are outside the application's control and thus out of scope for strict UI requirements

## 3. Functional Requirements

### 3.1 Notation and Structure

Each functional requirement is specified with:

- Requirement ID: FR--
- Feature Name
- Description

- Preconditions
- User Actions
- Expected System Behavior
- Validation and Error Handling

For brevity, requirements are grouped by feature/module. Within each module, requirements are atomic and testable (except where explicitly describing inherently probabilistic outcomes like randomness).

### 3.2 Global / Cross-Page Requirements

#### FR-G-01 – Home Page – Examples Listing

- Feature Name: Home Page Examples List
- Description: Display a list of links to all available example pages
- Preconditions: User accesses [the-internet.herokuapp.com](https://the-internet.herokuapp.com)
- User Actions:
  - Load the root URL
- Expected System Behavior:
  - The page shall display a heading "Welcome to the-internet" and a subheading "Available Examples"
  - The page shall present a vertical list of example names, each rendered as a hyperlink
  - Each hyperlink text shall match the example name (e.g., "Checkboxes", "File Upload", "Typos")
- Validation / Error Handling:
  - There is no explicit validation; the page must render successfully without requiring input

#### FR-G-02 – Global Footer

- Feature Name: Footer Attribution
- Description: Provide a consistent footer section attributing the site to Elemental Selenium on most example pages
- Preconditions: User navigates to an example page (e.g., /checkboxes, /upload, /dropdown, /inputs, /download)
- User Actions:
  - Scroll to the bottom of the example page
- Expected System Behavior:
  - The page shall display a footer containing the text "Powered by Elemental Selenium"

- Validation / Error Handling:
  - No user input; footer rendering must not interfere with the primary feature

### 3.3 Selected Feature Modules

Below, detailed requirements are specified for representative and higher-interaction modules. Remaining modules are covered in section 3.4 at a concise but still precise level.

#### 3.3.1 Checkboxes (/checkboxes)

Source text: "Checkboxes checkbox 1 checkbox 2 Powered by Elemental Selenium"

FR-CB-01 – Rendering of Checkboxes

- Description: Display two checkboxes with labels
- Preconditions: User navigates to /checkboxes
- User Actions:
  - Load /checkboxes
- Expected Behavior:
  - The page shall display at least two checkbox input controls, labeled "checkbox 1" and "checkbox 2" or equivalent nearby text
- Validation / Error Handling:
  - The page shall load without requiring user input

FR-CB-02 – Checkbox State Toggle

- Description: Allow the user to toggle checkbox states
- Preconditions: /checkboxes is loaded and at least one checkbox is visible
- User Actions:
  - Click a checkbox
- Expected Behavior:
  - On click, the target checkbox shall toggle its checked/unchecked state according to browser default behavior
- Validation / Error Handling:
  - No explicit error messages. Browser default behavior applies

#### 3.3.2 Form Authentication / Login (/login)

Source text: "This is where you can log into the secure area. Enter tomsmith for the username and SuperSecretPassword! for the password. If the information is wrong you should see error messages. Username, Password, Login."

FR-FA-01 – Login Form Rendering

- Description: Display username and password fields with a Login button

- Preconditions: User navigates to /login
- User Actions:
  - Load /login
- Expected Behavior:
  - The page shall display a heading indicating it is a "Login Page"
  - The page shall display an input field labeled "Username"
  - The page shall display an input field labeled "Password"
  - The page shall display a "Login" button
- Validation / Error Handling:
  - No validation on initial page load

#### FR-FA-02 – Successful Login with Valid Credentials

- Description: Grant access to a secure area when valid credentials are provided
- Preconditions: /login is loaded
- User Actions:
  - Enter "tomsmith" in the Username field
  - Enter "SuperSecretPassword!" in the Password field
  - Click the "Login" button
- Expected Behavior:
  - The system shall authenticate the user as successful when the username is "tomsmith" and password is "SuperSecretPassword!"
  - Upon success, the system shall navigate to a "secure area" page (e.g., via URL change /secure or equivalent) that indicates successful login
- Validation / Error Handling:
  - No error message shall be displayed for the above valid credentials

#### FR-FA-03 – Error Handling for Invalid Credentials

- Description: Display error messages when login information is incorrect
- Preconditions: /login is loaded
- User Actions:
  - Enter any username and/or password combination other than the known valid tomsmith / SuperSecretPassword!
  - Click the "Login" button
- Expected Behavior:

- The system shall display an error message indicating that the information is wrong
  - The system shall not navigate to the secure area page
- Validation / Error Handling:
  - The error message shall be visually associated with the login form (exact text is not constrained beyond indicating wrong information)

#### **4. Non-Functional Requirements**

- **4.1 Usability**
  - NFR-U-01: Page structures shall be simple and minimalistic so that intended test interactions (clicks, key presses, hovers, etc.) are easily discoverable without prior instruction
  - NFR-U-02: Each example page shall include explanatory text at or near the top that clearly states the behavior being demonstrated
  - NFR-U-03: Important interactive controls (such as "Login", "Add Element", "Where am I?", "Click for JS Alert") shall be visually distinguishable as buttons or links and labeled with descriptive text reflecting their actions